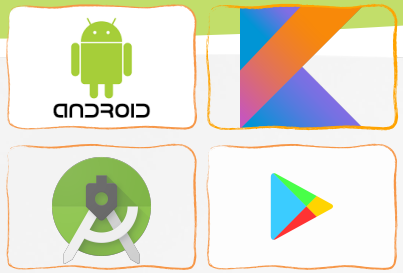
Software Development for Mobile Devices

Firebase Cloud Messaging

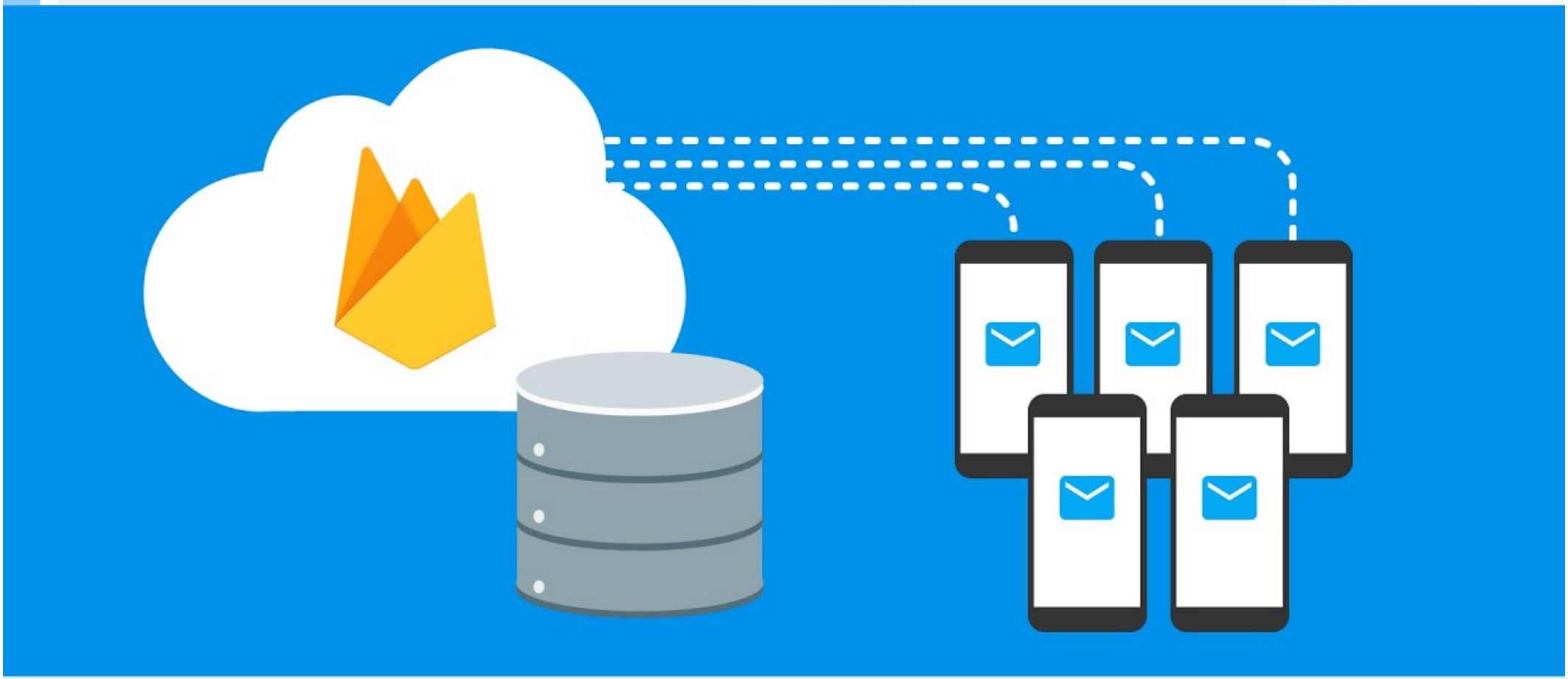
Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

Content



- **Introduction**
- FCM registration token management
- Firebase Cloud Messaging on Android

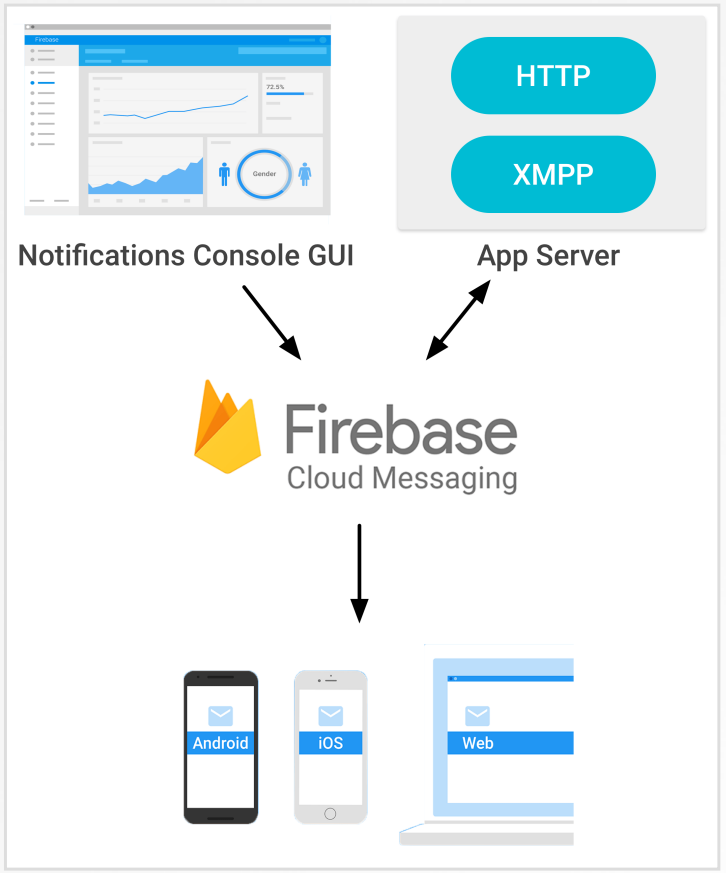
Firebase Cloud Messaging



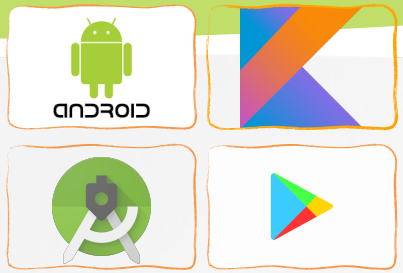
Cloud Messaging

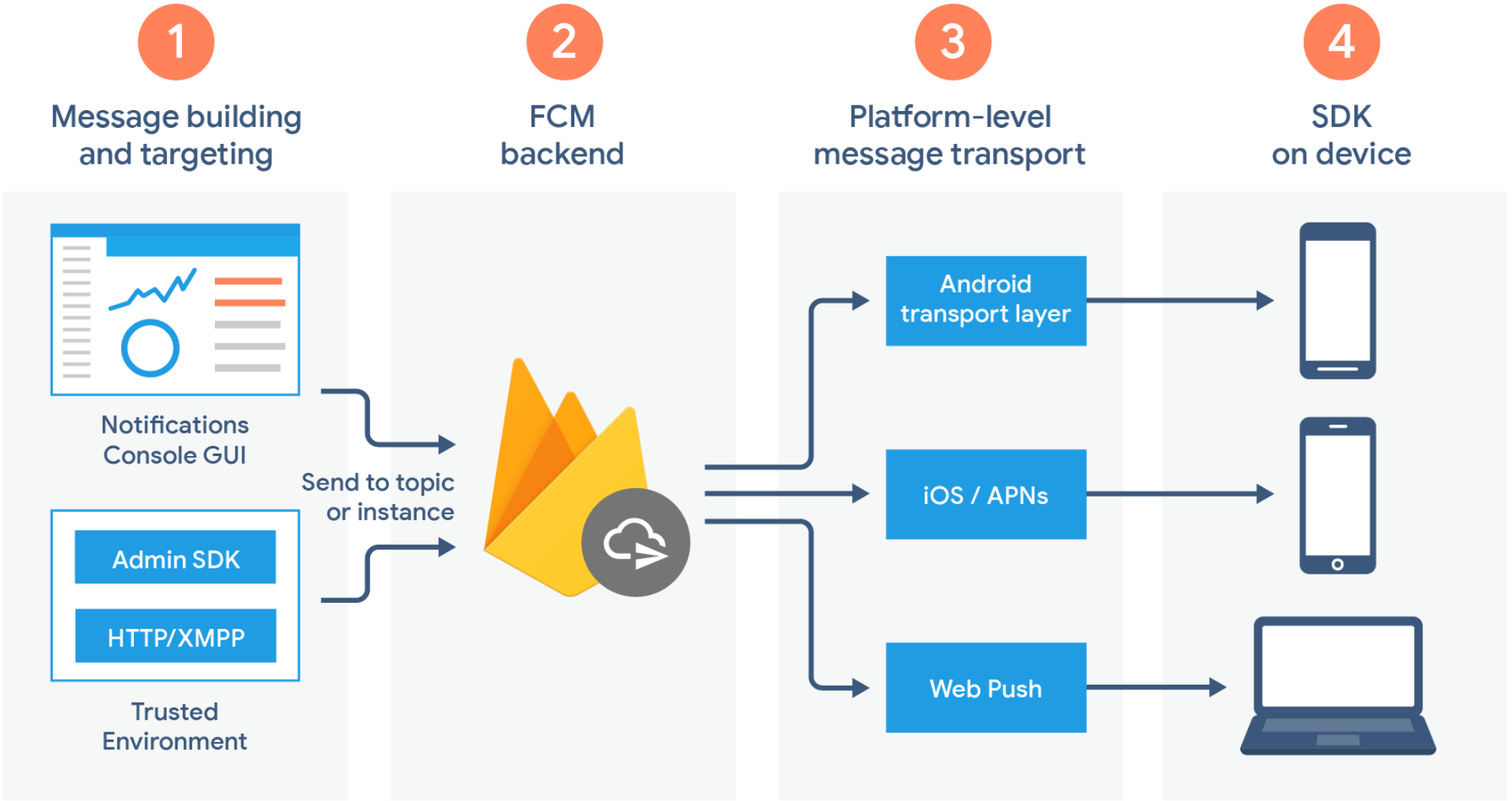
Firebase Cloud Messaging

- Firebase Cloud Messaging (FCM) is a cross-platform messaging solution that lets you reliably send messages at no cost
  - Notify a client app that new email or other data is available to sync.
  - For instant messaging, a message can transfer a payload of up to 4000 bytes to a client app.

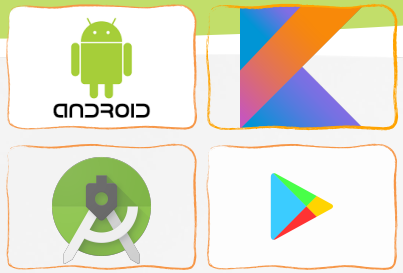


FCM Architectural Overview



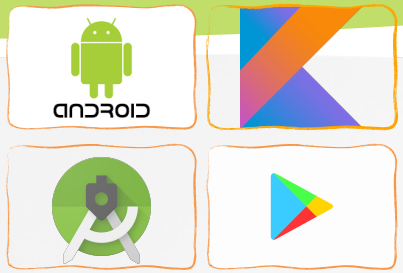


FCM Lifecycle flow



- Register devices to receive messages from FCM
  - An instance of a client app registers to receive messages, obtaining a ***registration token*** that uniquely identifies the app instance.
- Send a message. The app server sends messages to the client app:
  - The message is composed with Notifications composer or trusted environment
  - Then we send that message request to the FCM backend.
  - The FCM backend receives the message request, generates a message ID and other metadata, and sends it to the platform specific transport layer.
  - When the device is online, the message is sent via the platform-specific transport layer to the device.
  - On the device, the client app receives the message or notification

Notification messages



- We can send notification messages using the Firebase console for testing or for marketing and user re-engagement.
- To programmatically send notification messages using the Admin SDK or the FCM protocols, set the notification key with the necessary predefined set of key-value options for the user-visible part

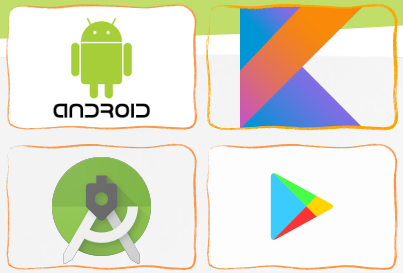
```
{
  "message":{
    "token":"bk3RNwTe3H0:CI2k_HHwgIpoDKCIZvvDMExUdFQ3P1...",
    "notification":{
      “title":"New Booking for you",
      “body":"You have new booking at 3:30PM”
    }
  }
}
```

Data messages

- Data messages: set the appropriate key with your custom key-value pairs to send a data payload to the client app
  - *Do not use any reserved words in your custom key-value pairs: “from”, "notification," "message_type", or any word starting with "google" or “gcm."*

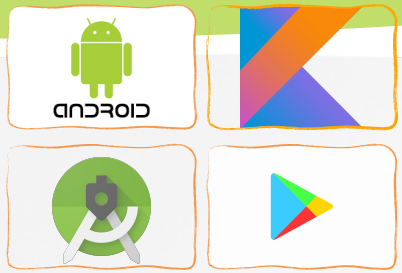
```
{
  "message":{
    “token":abk3RNwTe3H0:CI2k_HHwgIpoDKCIZvvDMExUdFQ3P1...",
    "notification":{
      “title":"New Booking for you",
      “body":"You have new booking at 3:30PM”
    }
    "data":{
      “center_id”: “111”,
    }
  }
}
```

Content



- Introduction
- **FCM registration token management**
- Firebase Cloud Messaging on Android

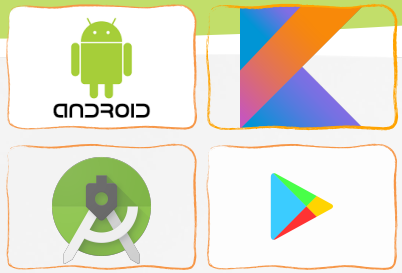
Best practices for FCM registration token management



We will waste resources by sending messages to inactive devices with stale registration tokens:

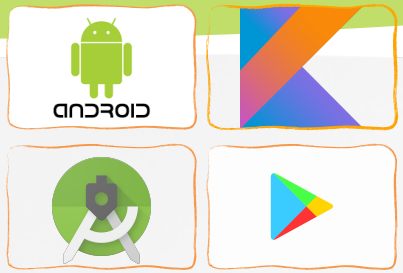
- **Store registration tokens on your server** and **keep** an updated list of active tokens.
  - Implementing a token timestamp in your code and your servers, and updating this timestamp at regular intervals whenever it changes
    - The app is restored on a new device
    - The user uninstalls/reinstall the app
    - The user clears app data
- **Remove stored tokens that become stale (~2 months), invalid**
  - With the HTTP v1 API, these error messages may indicate that your send request targeted stale or invalid tokens
    - UNREGISTERED (HTTP 404)
    - INVALID_ARGUMENT (HTTP 400)

Best practices for FCM registration token management



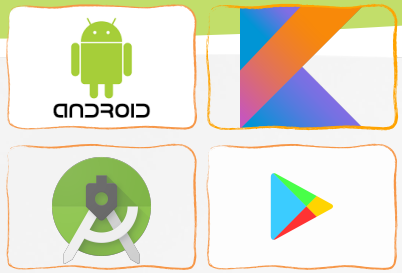
- Unsubscribe stale tokens from topics
  - Our app should resubscribe to topics once per month and/or whenever the registration token changes.
    - It's a self-healing solution, where the subscriptions reappear automatically when an app becomes active again.
  - If an app instance is idle for 2 months, we should unsubscribe it from topics using the Firebase Admin SDK to delete the token/topic mapping from the FCM backend.

Content



- Introduction
- FCM registration token management
- **Firebase Cloud Messaging on Android**

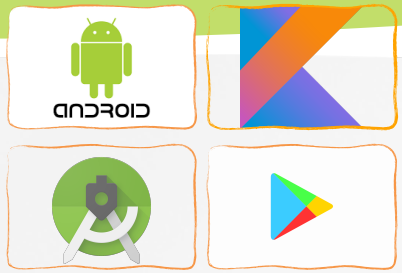
Firebase Cloud Messaging on Android



- FCM clients require:
  - Devices with Android 4.4 or higher that also have the Google Play Store app installed.
  - Emulator with Android 4.4 with Google APIs.
- Main steps for using FCM:
  - Register your app with Firebase on Firebase console.
  - Add a Firebase configuration file with ***google-services.json***
  - ***Add Firebase SDKs to your app***

```
dependencies {
    // Import the Firebase BoM
     implementation platform('com.google.firebase:firebase-bom:29.3.0')
     implementation ‘com.google.firebase:firebase-analytics'

     implementation 'com.google.firebase:firebase-messaging-ktx'
}
```

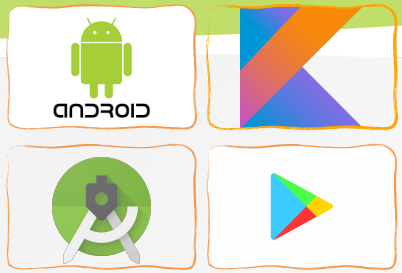
Firebase Cloud Messaging on Android



- Edit your app manifest
  - Add a service that extends FirebaseMessagingService.
    - It is required if you want to do any message handling beyond receiving notifications on apps in the background. To receive notifications in foregrounded apps, to receive data payload, to send upstream messages…
      - *MyFirebaseMessagingService* is your Kotlin class

```
<application>
  <!--   …   -->
  <service
      android:name=".MyFirebaseMessagingService"
      android:exported="false">
      <intent-filter>
          <action android:name="com.google.firebase.MESSAGING_EVENT" />
      </intent-filter>
  </service>
</application>
```

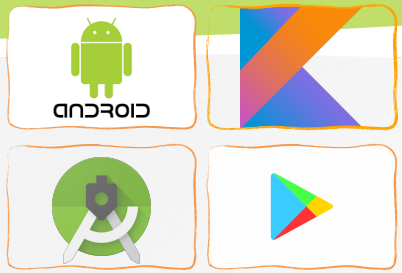
Firebase Cloud Messaging on Android



- On initial startup, the FCM SDK generates a ***registration token*** for the client app instance.
- If we want to target single devices or create device groups, we'll need to access this token by extending ***FirebaseMessagingService*** and overriding ***onNewToken***.
- The registration token may change when:
  - The app is restored on a new device
  - The user uninstalls/reinstall the app
  - The user clears app data

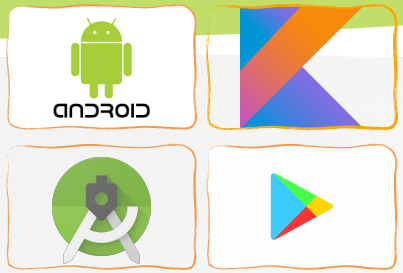
```
class MainActivity : AppCompatActivity() {
    lateinit var statusTv : TextView

    companion object {
        private const val TAG = "FromMainActivity"
    }
```

Firebase Cloud Messaging on Android



```
  override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      setContentView(R.layout.activity_main)

      val firebaseTokenBtn : Button = findViewById(R.id.firebaseTokenBtn)
      firebaseTokenBtn.setOnClickListener {

    val firebaseTokenBtn : Button = findViewById(R.id.firebaseTokenBtn)
      firebaseTokenBtn.setOnClickListener {
          Firebase.messaging.getToken().addOnCompleteListener { task ->
              if (!task.isSuccessful) {
                  Log.i(TAG, "Fetching FCM registration token failed",
                 task.exception)
              }

              // Get new FCM registration token
              val token = task.result
              statusTv.append("\n${token}")
          }
      }
  }
}
```

Firebase Cloud Messaging on Android

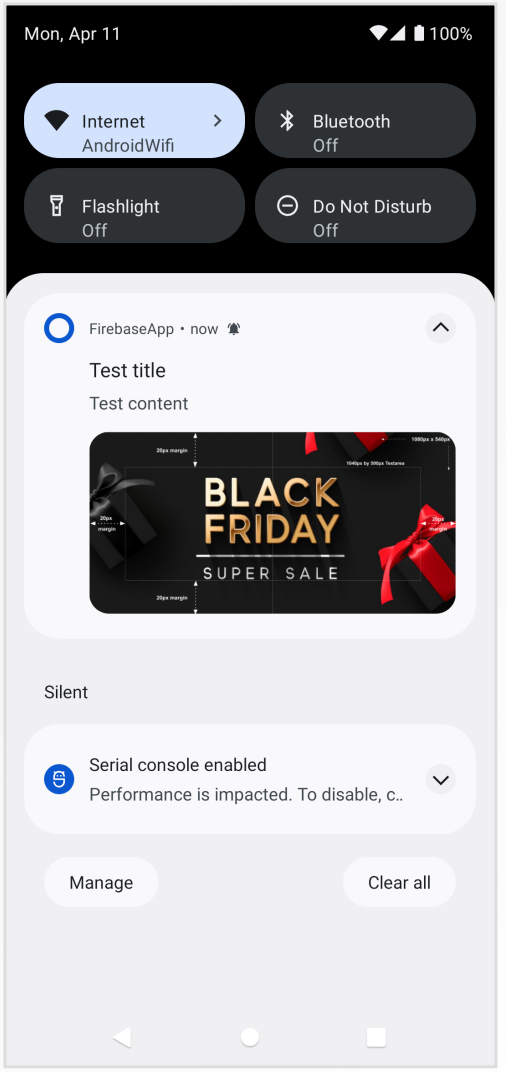


- Monitor token generation
  - ***onNewToken*** : fires whenever a new token is generated

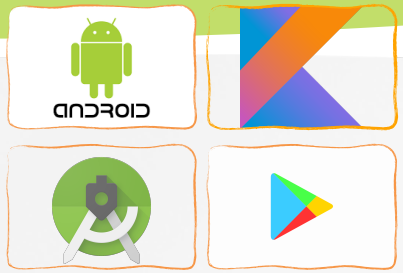
```
class MyFirebaseMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) {
        Log.d(TAG, "Refreshed token: $token")

        // If you want to send messages to this application instance or
        // manage this apps subscriptions on the server side, send the
        // FCM registration token to your app server.
        //  sendRegistrationToServer(token)
    }
    companion object {
        private const val TAG = "MyFirebaseMsgService"
    }
}
```

Firebase Cloud Messaging on Android

- Now we will send a test notification message from the Notifications composer to a development device when the app is in the background
  - Open the Notifications composer and select **New notification**.
  - Enter the Notification title.
  - Enter the Notification text.
  - Enter the Notification image (optional): <1MB
  - Select Send test message.
  - In the field labeled **Add an FCM registration token**, enter the registration token you obtained in a previous steps.
  - Click **Send test mesage**



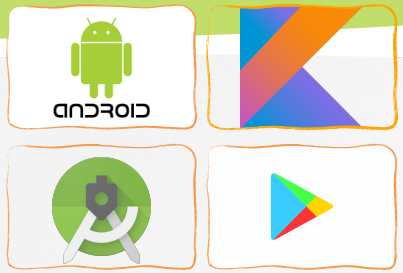
Firebase Cloud Messaging on Android



- If your serve sends topic messages using the Admin SDK or REST API for FCM, we need to subscribe the client app to that topic
- Client apps can subscribe to any existing topic, or they can create a new topic which will be created in FCM and any client can subsequently subscribe to it.
- To subscribe to a topic, the client app calls Firebase Cloud Messaging ***subscribeToTopic()*** with the FCM topic name.

```
Firebase.messaging.subscribeToTopic("weather")
        .addOnCompleteListener { task ->
            var msg = getString(R.string.msg_subscribed)
            if (!task.isSuccessful) {
                msg = getString(R.string.msg_subscribe_failed)
            }
            Log.d(TAG, msg)
        }
```

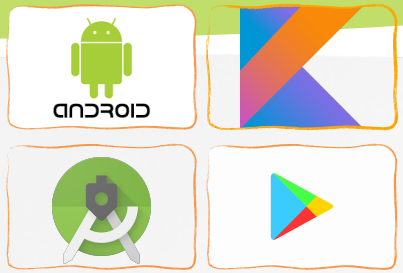
Firebase Cloud Messaging on Android



- Receive and handle topic messages
  - Use a service that extends FirebaseMessagingService.
  - Override ***onMessageReceived***
- When the notification is **delivered when your app is in the background,** it's delivered to the device's system tray:
  - N**otification messages:** user tap on a notification opens the app launcher by default.
  - **Messages with both notification and data payload:** the data payload is delivered in the extras of the intent of your launcher Activity.

App state	Notification	Actions
Foreground	onMessageReceived	onMessageReceived
Background	System tray	Notification: system tray Data: in extras of the intent.

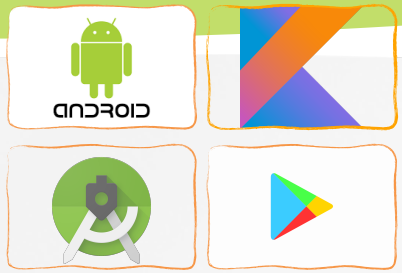
Firebase Cloud Messaging on Android



- Handle notification messages in a foregrounded app

```
override fun onMessageReceived(remoteMessage: RemoteMessage) {
    Log.d(TAG, "From: ${remoteMessage.from}")

    // Check if message contains a data payload.
    if (remoteMessage.data.isNotEmpty()) {
        Log.d(TAG, "Message data payload: ${remoteMessage.data}")
        if (/* Check if data needs to be processed by long running job */ true) {
            // For long-running tasks (10s or more) use WorkManager.
            scheduleJob()
        } else {
            // Handle message within 10 seconds
            handleNow()
        }
    }
    // Check if message contains a notification payload.
    remoteMessage.notification?.let {
        Log.d(TAG, "Message Notification Body: ${it.body}")
    }

    //Generate your own notification as a result of a received FCM

    sendNotification(remoteMessage.notification?.body ?: "empty_body")
}
```

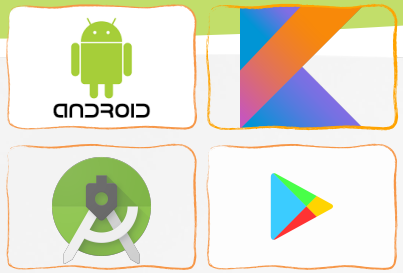
Firebase Cloud Messaging on Android



- Handle notification messages in a foregrounded app

```
/**
 * Schedule async work using WorkManager.
 */
private fun scheduleJob() {
    val work = OneTimeWorkRequest.Builder(MyWorker::class.java).build()
    WorkManager.getInstance(this).beginWith(work).enqueue()
}

/**
 * Handle time allotted to BroadcastReceivers.
 */
private fun handleNow() {
    Log.d(TAG, "Short lived task is done.")
}
```

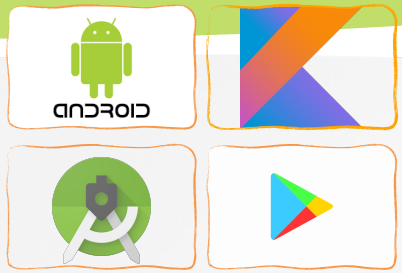
Firebase Cloud Messaging on Android



- Handle notification messages in a foregrounded app

```
private fun sendNotification(messageBody: String) {
    //MainActivity is the activity will be show when you tap this notification
    // you can change it with your own activity
    val intent = Intent(this, MainActivity::class.java)
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)
    val pendingIntent = PendingIntent.getActivity(this, 0 /* Request code */,
        intent, PendingIntent.FLAG_IMMUTABLE)

    val default_notification_channel_id = "default_notification_channel_id"
    val fcm_message = "fcm_message title"
    val defaultSoundUri =
RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION)
    val notificationBuilder = NotificationCompat.Builder(this,
        default_notification_channel_id)
        .setContentTitle(fcm_message).setContentText(messageBody)
        .setAutoCancel(true).setSound(defaultSoundUri)
        .setContentIntent(pendingIntent)
        .setSmallIcon(com.facebook.share.R.drawable.abc_btn_default_mtrl_shape)
```

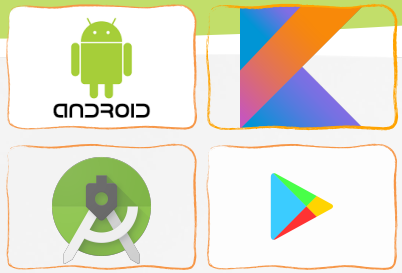
Firebase Cloud Messaging on Android



- Handle notification messages in a foregrounded app

```
   val notificationManager =
getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    // Since android Oreo notification channel is needed.
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(default_notification_channel_id,
            "Channel human readable title",
            NotificationManager.IMPORTANCE_DEFAULT)
        notificationManager.createNotificationChannel(channel)
    }

    notificationManager.notify(0/*ID of notification*/,
        notificationBuilder.build())
}
```

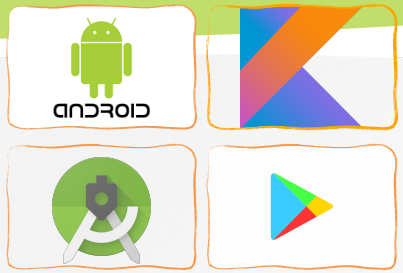
Firebase Cloud Messaging on Android



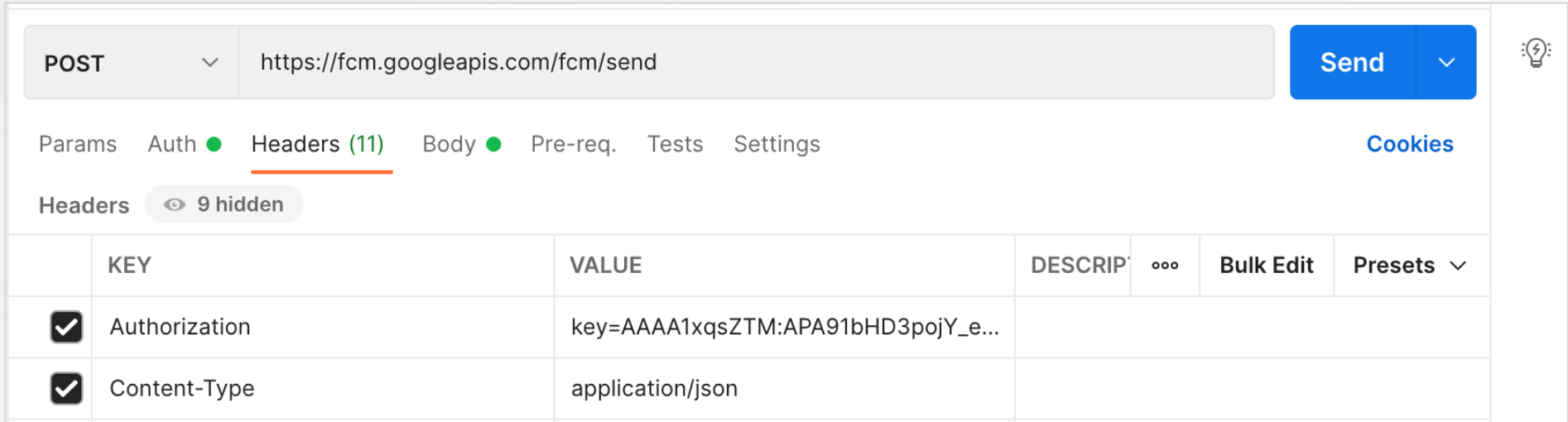
- Handle notification messages in a backgrounded app
- When your app is in the background, Android directs notification messages to the system tray. A user tap on the notification opens the app launcher by default.
  - This includes messages that contain both notification and data payload which is delivered in the extras of the intent of your launcher Activity.

```
override fun onStart() {
    super.onStart()

    val val1 = intent.getStringExtra("key1")
    val extraData = intent.extras
    Log.d(TAG, val1.toString())
    Log.d(TAG, extraData.toString())
}
```

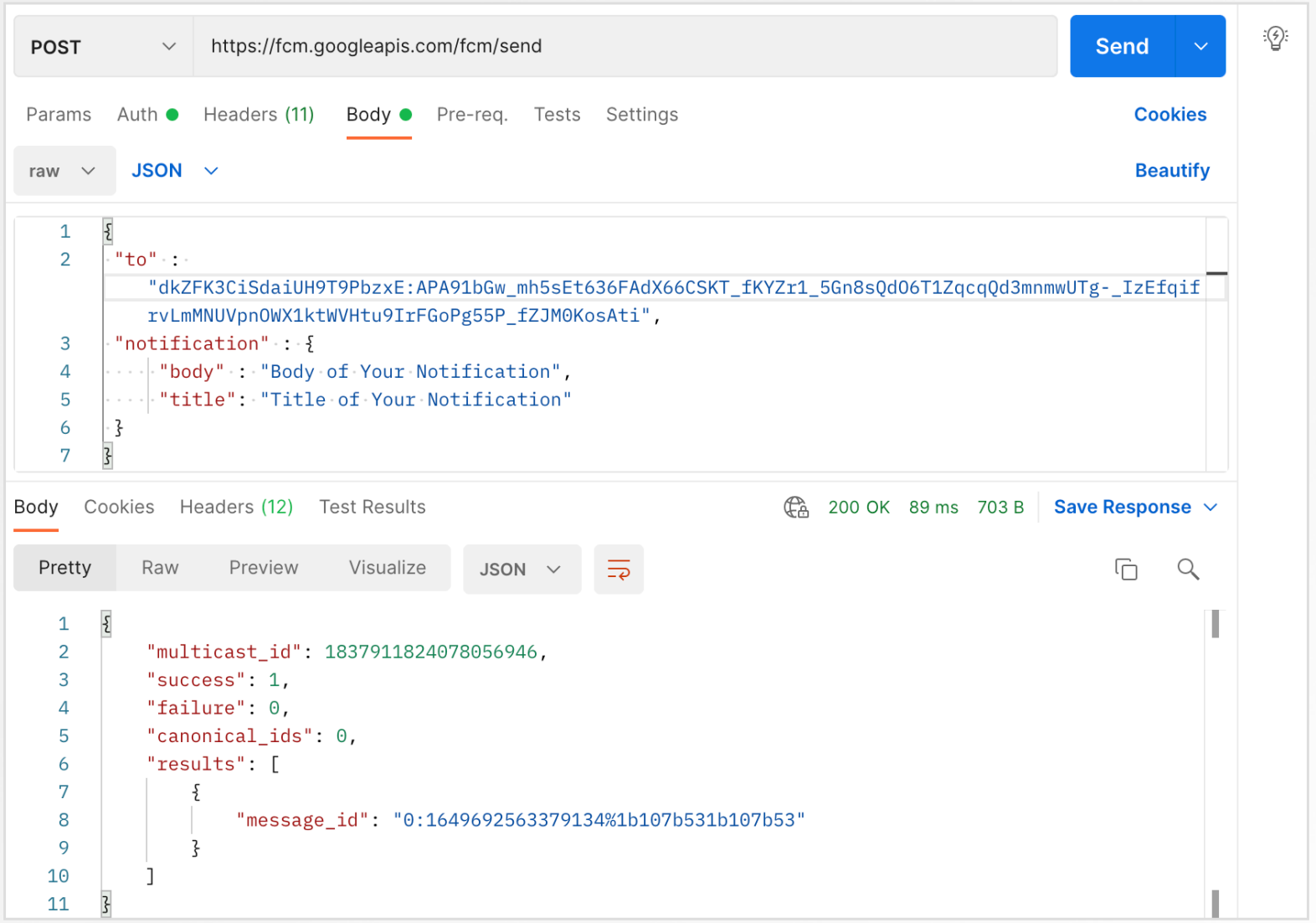
Test FCM Notification with Postman



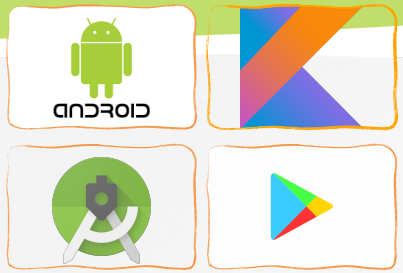
- *Copy <**Server Key**> from* Firebase Console → Project Settings → Cloud Messaging
- Add the following details in the Postman
  - **Endpoint** : https://fcm.googleapis.com/fcm/send
  - **Method** : POST
  - **Headers**
    - **Authorization** : key=[server_key]
    - **Content-Type** : application/json



Test FCM Notification with Postman



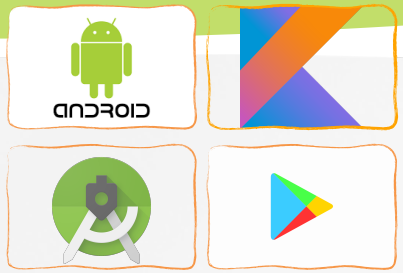
Practice



- Try to send messages with the Notifications composer and check notification on your device

Q&A

Reference



- https://firebase.google.com/docs/cloud-messaging
- https://medium.com/@mountainappstudio/open-specific-activity-when-notification-clicked-in-fcm-android-app-development-f1c1d9a75fb5
- Peter Späth, Learn Kotlin for Android Development, 2019
- Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- Peter Späth, Pro Android with Kotlin, 2019
- Milos Vasic, Mastering Android Development with Kotlin, 2017